WORKSHOP OUTLINE

Build Your AI Project Manager

11 November 2026 · ~2 hours · hands-on, code · CrewAI + local Ollama · senior / 5+ yrs

- **Goal:** Senior engineers build an agentic PM assistant — and understand how agents work under the hood.
- **Use case:** An AI project manager: a brief in, a task breakdown + risks + status update out.
- **Audience:** Senior, 5+ years. Comfortable with Python; no prior agent experience needed.
- **Format:** ~2 hours. Presenter live-codes; audience follows with the code bundle. Online session; use your own laptop with Ollama set up beforehand.
- **They leave with:** Runnable code (pm_crew.py, tool_demo.py, virtual_office.py) and a build guide.

The arc: See it → How agents work → Build it in code → A virtual office → Make it real

Session flow (120 minutes)

Time	Section (slides)	What happens
0:00 8 min	**Welcome & your guide** Slides 1–3	Intro, speaker background (production agent systems), agenda, and the promise: you'll build a working agentic PM assistant.
0:08 5 min	**What we're building** Slide 4	Show the finished assistant turning a brief into tasks, risks, and a status update — running locally. Anchors the build.
0:13 25 min	**01 — How agents work** Slides 5–9	Agent anatomy (LLM + tools + memory + loop); the ReAct loop; single agent vs. a crew (sequential / hierarchical); the open-source local stack (CrewAI → Ollama).
0:38 45 min	**02 — Build the PM assistant** Slides 10–17	Live-coded: setup → first agent → a tool → typed (Pydantic) output → two specialists → run the sequential crew on a real brief. **Hands-on:** *audience follows along from the code bundle; everyone runs the final crew.*
1:23 15 min	**Intermezzo — the virtual office** Slides 18–21	From pipeline to office: a Project Lead returns a typed work plan and Python dispatches it to CV / Backend / Frontend / QA engineers. Discuss the power vs. cost/latency trade-off, and why Process.hierarchical did not delegate on these local models.

		Hands-on: *run virtual_office.py.*
1:38 15 min	**03 — Make it real** Slides 22–25	Guardrails + human-in-the-loop; observability and cost with local models; evaluation; responsible, least-privilege agents and PDP-Law-aware data handling.
1:53 7 min	**Recap & discussion** Slides 26–28	The whole build in seven lines; discussion — 'what project chore would you hand an agent crew first?'; close and Q&A.

What participants need (critical for the build block)

- A laptop that can run Ollama — 8 GB RAM minimum, 16 GB recommended; Apple Silicon or a discrete GPU is a big plus.
- Ollama installed and llama3.1 PULLED BEFORE the session — the ~4.7 GB download can take hours on a slow connection.
- Python 3.10–3.13 (not 3.14) and 'pip install -r requirements.txt' done beforehand; 'python check_setup.py' passes.
- The code bundle (pm_crew.py, tool_demo.py, virtual_office.py, check_setup.py, brief.md, requirements.txt) downloaded in advance.

Facilitator notes

- Send the setup instructions + model pull at least 3 days ahead. Treat 'Ollama installed and model pulled' as the entry ticket.
- Presenter live-codes; participants follow and run — do not wait for everyone to type every line.
- Have the finished code ready to paste, and a pre-recorded run as a fallback if laptops are too slow.
- Hard stop after the PM crew runs (slide 17). If behind, shorten the intermezzo — keep the core build and the 'make it real' safety section.
- Local-model latency is the main risk: suggest a small model (llama3.2:3b) for weaker laptops.

Prepared by Felix Yustian Setiono for the MurzEdu Program (ecommurz). Timings are a guide — adjust to the group and the hardware.